

The Complete AI Ecosystem Map

Every component, its role, what breaks without it, and how everything connects

00 How to Read This Guide

The AI ecosystem has 12 layers — from raw compute at the bottom to user-facing applications at the top. Each layer depends on the one below it. This guide explains every layer: what it does, what role it plays, and what you simply cannot build without it.

Read top-to-bottom for a mental model of the full stack. Use the tables as a reference when you encounter a new tool and want to understand where it fits.

THE BIG PICTURE

User Query → App Layer → Agent/Orchestration → LLM Brain → Tools/Memory → Vector Store → Embeddings → Chunking → Raw Data Every arrow is a dependency. Remove any layer and the system breaks in a specific, diagnosable way.

01 Layer 1 — Compute Infrastructure (GPUs / TPUs)

All AI models — training and inference — run on specialized hardware. CPUs are too slow for the parallel matrix multiplications that neural networks require. GPUs (Graphics Processing Units) were repurposed for AI because they have thousands of cores that can multiply matrices in parallel.

Hardware	Provider	Role	Used For
NVIDIA A100 / H100	NVIDIA	Training large models	GPT-4, Llama, Claude training. ~\$30K/card.
NVIDIA RTX 4090	NVIDIA	Local inference / fine-tuning	Run 7B-70B models locally. ~\$2K.
Google TPU v5	Google	Training at Google scale	Gemini training. TPUs are custom ASICs for tensor ops.
AWS Inferentia	AWS	Cost-optimized inference	Deploy models cheaply on AWS without GPU.
Apple M-series	Apple	Local inference on Mac	Unified memory architecture — run 7B-13B locally.
CPU (x86/ARM)	Intel/AMD/ARM	Small model inference	Tiny models, embedding, classical ML.

WHAT YOU CANNOT DO WITHOUT COMPUTE

Train any model. Run models larger than CPU memory allows. Achieve production-grade inference latency. Everything in AI ultimately runs on compute — it is the foundational dependency.

02 Layer 2 — Foundation Models (LLMs, Vision, Audio)

Foundation models are large neural networks pre-trained on massive datasets. They are the "brains" of the AI ecosystem — the reasoning, generation, and understanding engines that everything else is built on top of.

Language Models (LLMs)

Model	Provider	Open?	Context	Strengths
GPT-4o	OpenAI	No	128K	General purpose, vision, function calling, best ecosystem
GPT-4o-mini	OpenAI	No	128K	Cost-efficient, 90% quality, ideal for RAG generation
Claude 3.5 Sonnet	Anthropic	No	200K	Best coding + analysis, safety, largest context, long-doc RAG
Claude 3 Haiku	Anthropic	No	200K	Fast and cheap — high-volume RAG, classification
Gemini 1.5 Pro	Google	No	1M	1M token context, multi-modal, best for huge documents
Llama 3.3 (70B)	Meta	Yes	128K	Best open-weight, run anywhere, no API cost
Llama 3.2 (1B/3B)	Meta	Yes	128K	Edge deployment, mobile, embedded systems
Mistral 7B / 8x7B	Mistral	Yes	32K	Efficient, strong coding, MoE architecture
Qwen 2.5 / Coder	Alibaba	Yes	128K	Best open code model, multilingual, your internal LLM
Phi-3 / Phi-4	Microsoft	Yes	128K	Small but surprisingly capable. Edge/local inference.
Gemma 2 (9B/27B)	Google	Yes	8K	Google open-weight. Strong for fine-tuning.
DeepSeek V3/R1	DeepSeek	Yes	64K	Strong reasoning, competitive with GPT-4, cheap training

WHAT YOU CANNOT DO WITHOUT LLMS

Generate natural language. Reason over retrieved context. Understand intent from ambiguous queries. Plan multi-step tasks. Write code. Extract structured data from unstructured text. LLMs are the reasoning core of every AI application.

Embedding Models (Specialist Models)

Separate from LLMs, embedding models are small, fast models trained specifically to convert text into vectors for similarity search. They are the backbone of RAG retrieval.

Model	Dims	Type	Best For
-------	------	------	----------

all-MiniLM-L6-v2	384	Dense (local)	Fast local RAG, prototyping
BGE-M3	1024	Dense+Sparse (local)	Multi-lingual, hybrid search, production OSS
E5-large-instruct	1024	Dense (local)	Instruction-tuned, strong benchmarks
text-embedding-3-small	1536	Dense (API)	OpenAI, balanced cost/quality
text-embedding-3-large	3072	Dense (API)	Highest OpenAI quality
Cohere embed-v3	1024	Dense (API)	Multi-lingual, int8/binary compression
SPLADE++	Sparse	Sparse (local)	Neural sparse — BM25 quality + semantic expansion
ColBERT v2	Per-token	Late interaction	Highest accuracy, more storage, research/premium RAG

WHAT YOU CANNOT DO WITHOUT EMBEDDING MODELS

Semantic search. RAG retrieval. Duplicate detection. Clustering documents by topic. Recommendation systems. Without embeddings, you are limited to exact keyword matching only.

Vision and Multi-Modal Models

Model	Provider	Capability
GPT-4o Vision	OpenAI	Image understanding, OCR, diagram analysis in chat context
Claude 3.5 Sonnet Vision	Anthropic	Document images, charts, UI screenshots, complex visual reasoning
Gemini 1.5 Pro Vision	Google	Video understanding, image analysis, multi-modal long context
LLaVA / LLaVA-Next	Open-source	Local vision-language model. Run on your own GPU.
ColPali	Open-source	Embed document PAGE IMAGES directly — no OCR needed for RAG
CLIP / SigLIP	OpenAI / Google	Cross-modal embeddings: images and text in same vector space
DALL-E 3	OpenAI	Text → image generation via API
Stable Diffusion 3	Stability AI	Open-source text → image. Run locally with diffusers.
Whisper	OpenAI	State-of-the-art speech-to-text. Local or API. 99 languages.
ElevenLabs	ElevenLabs	Text-to-speech with voice cloning. Production audio.

03 Layer 3 — Tokenization and Context Management

Before text reaches an LLM, it must be converted to token IDs. Context management determines what fits in the model's memory window and how it is structured.

Component	Role	Key Tools / Libraries
Tokenizer	Converts text ↔ token IDs. Each LLM has its own vocabulary.	tiktoken (OpenAI), HuggingFace tokenizers, SentencePiece

Context Window Manager	Tracks tokens used, truncates when limit reached	LangChain ConversationBufferWindowMemory, custom trimming
Prompt Template	Structures system prompt + history + context + query	LangChain PromptTemplate, Jinja2, f-strings
Message Formatter	Formats chat history in model-specific format	ChatPromptTemplate, apply_chat_template() in transformers
Context Compression	Summarizes or filters context when window is full	LLMLinguaCompressor, ContextualCompressionRetriever

WHAT YOU CANNOT DO WITHOUT CONTEXT MANAGEMENT

Handle conversations longer than the context window. Efficiently use token budget (irrelevant context wastes tokens and hurts quality). Build stateful multi-turn applications.

04

Layer 4 — Data Layer (Storage, Vector DBs, Traditional DBs)

AI applications need multiple types of storage for different purposes. Understanding which storage to use for what is critical for building production systems.

Vector Databases — Semantic Search at Scale

Database	Type	Index	Hybrid?	Best For
Chroma	OSS / Local	HNSW	No	Prototyping, local dev, simplest setup
FAISS	Library	All	No	Custom pipelines, research, offline batch
Qdrant	OSS / Cloud	HNSW	Yes	Production hybrid search, fast, Rust-based
Weaviate	OSS / Cloud	HNSW	Yes	Multi-modal, GraphQL, strong ecosystem
Milvus / Zilliz	OSS / Cloud	HNSW/IVF	Limited	Billion-scale, enterprise
pgvector	PostgreSQL ext	HNSW/IVF	No	Already using Postgres — simplest upgrade
Pinecone	SaaS	Proprietary	Yes	Managed serverless, zero ops, fast
Supabase (pgvector)	SaaS	HNSW	No	Full-stack with auth, storage + vectors
Redis (VectorDB)	OSS/Cloud	HNSW	No	Low-latency, caching + vector in one store
OpenSearch/Elastic search	OSS/Cloud	HNSW	Yes	Existing search infra + vector upgrade

WHAT YOU CANNOT DO WITHOUT VECTOR DBS

Semantic search at scale. RAG retrieval for large knowledge bases. Persistent embeddings across sessions. Hybrid keyword+semantic search. Without a vector DB, you must re-embed everything on every query — completely impractical.

Traditional Databases — Structured Knowledge

Type	Examples	Role in AI Systems
Relational (SQL)	PostgreSQL, MySQL, SQLite	Store user data, sessions, metadata, audit logs, config
Document (NoSQL)	MongoDB, Firestore	Store conversation history, agent state, flexible schemas
Key-Value	Redis, DynamoDB	Cache embeddings, session state, rate limiting, fast lookups
Object Storage	S3, GCS, Azure Blob	Store raw documents, model checkpoints, large files
Graph DB	Neo4j, Amazon Neptune	Knowledge graphs, entity relationships for advanced RAG
Time Series	InfluxDB, TimescaleDB	Store model metrics, latency, cost tracking over time

Document Processing Tools

Tool	Language	Role
PyMuPDF (fitz)	Python	Fast PDF text + metadata extraction. Best for structured PDFs.
pdfplumber	Python	PDF text + table extraction. Great for tabular PDFs.
Unstructured.io	Python/API	Universal document parser — PDFs, DOCX, HTML, images. Production-grade.
Docling (IBM)	Python	Advanced PDF parsing with layout understanding. New 2024.
AWS Textract	API	OCR + form/table extraction from scanned documents. Best for KYC/compliance.
Azure Document Intelligence	API	Form recognition, invoice parsing, multi-language OCR.
LlamaParse	API	LLM-powered PDF parsing — handles complex layouts, tables, charts.
python-docx	Python	Parse and create Word documents.
BeautifulSoup / Playwright	Python	HTML parsing and JavaScript-rendered page scraping.

WHAT YOU CANNOT DO WITHOUT GOOD DOCUMENT PROCESSING

Load data into RAG accurately. Complex PDFs with tables, multi-columns, or images will produce garbage text if loaded with naive tools — corrupting every downstream stage.

05

Layer 5 — Chunking, Embedding & Indexing Pipeline

The pipeline that transforms raw documents into a searchable vector index. This is the "offline" or "ingestion" pipeline in RAG — run once when documents change.

Stage	Tool Options	What It Does
-------	--------------	--------------

Chunking	LangChain TextSplitters, LlamaIndex NodeParsers, custom	Split documents into retrieval units
Embedding	sentence-transformers, OpenAI API, Cohere API	Convert text chunks to dense vectors
Sparse Indexing	BM25S library, Elasticsearch, OpenSearch	Build inverted index for keyword search
Vector Indexing	FAISS, Qdrant, Weaviate, pgvector	Store and index dense vectors for ANN search
Metadata Storage	PostgreSQL, MongoDB, DynamoDB	Store chunk metadata: source, page, section, date
Pipeline Orchestration	LangChain, LlamaIndex, Prefect, Airflow	Schedule, parallelize and monitor ingestion

06

Layer 6 — Retrieval Layer

At query time, the retrieval layer finds the most relevant content for a given user query. This is the "online" part of RAG — executed on every user request.

Component	Role	Tools / Algorithms
Query Analysis	Understand and pre-process the query	Query expansion, intent classification, query rewriting
Sparse Retrieval	Keyword-based first-pass retrieval	BM25 via Elasticsearch, OpenSearch, BM25S
Dense Retrieval	Semantic similarity retrieval	ANN search in Qdrant/Weaviate/Pinecone via HNSW
Hybrid Fusion	Combine keyword + semantic results	RRF (Reciprocal Rank Fusion), linear score combination
Metadata Filtering	Pre-filter by structured attributes	Date range, source, department, document type
Re-ranking	Re-score top-N with a cross-encoder	Cohere Rerank, FlashRank, BGE-Reranker
Context Compression	Remove irrelevant parts from retrieved chunks	LLMLingua, ContextualCompressionRetriever

WHAT YOU CANNOT DO WITHOUT THE RETRIEVAL LAYER

Ground LLM answers in your specific knowledge base. The LLM cannot access your internal documents without retrieval. Without good retrieval, even the best LLM produces hallucinated answers.

07

Layer 7 — Orchestration Frameworks

Orchestration frameworks tie all the components together — loading documents, calling embeddings, querying vector stores, building prompts, calling LLMs, and post-processing outputs. They are the "glue" layer.

Framework	Focus	Strengths	Best For
-----------	-------	-----------	----------

LangChain	General AI apps	Largest ecosystem, most integrations, chains + agents + RAG	Most use cases — start here
LlamaIndex	Document RAG	Richer indexing, query engines, better structured data	Document-heavy RAG, complex retrieval
LangGraph	Stateful agents	Graph-based cyclic workflows, multi-agent state machines	Complex agent workflows with loops
Haystack	Pipelines	Production pipelines, modular, strong evaluation	Enterprise RAG with full MLOps
DSPy	Prompt optimization	Programmatic prompting, auto-optimize prompts via ML	When you want to treat prompts as code
Semantic Kernel	Enterprise .NET/Python	Microsoft ecosystem, Azure integration, plugins	Microsoft/Azure shops
AutoGen	Conversational agents	Multi-agent conversations, agents debate/collaborate	Research, complex reasoning tasks

WHAT YOU CANNOT DO WITHOUT ORCHESTRATION

Build anything beyond a single LLM API call. Chains, RAG pipelines, agents, memory, multi-step workflows — all require orchestration. Without it, you write everything from scratch.

08

Layer 8 — Agentic Layer (Agents, Tools, Memory, MCP)

Agents extend LLMs from passive responders to active participants that can plan, execute multi-step tasks, use tools, and maintain state across interactions.

Agent Frameworks

Framework	Type	Key Concept	Best For
LangChain Agents	Single/Multi	ReAct, OpenAI-functions, structured-chat	General-purpose agents
LangGraph	Multi-agent	Stateful graph with cycles and conditions	Complex workflows, human-in-loop
CrewAI	Multi-agent	Role-based crews with tasks and processes	Team simulation, pipeline tasks
AutoGen	Multi-agent	Conversational agents that chat with each other	Debate, code review, research
OpenAI Assistants	Single	Managed threads + tools + file search	Managed agent infra, quick start
Phidata / Agno	Multi-agent	Teams with memory and knowledge	Full-featured agent apps
Smolagents (HuggingFace)	Single/Multi	Minimal, code-first agents	Researchers, custom agent loops

Tool Ecosystem for Agents

Category	Tools	What Agents Can Do
Web Search	Tavily, Serper, Brave, DuckDuckGo	Research current events, find information
Code Execution	E2B, Daytona, PythonREPL, Modal	Write and run code, data analysis, math
Browser Automation	Playwright, Puppeteer, Browserbase, Stagehand	Navigate web, fill forms, scrape JS pages
File/Data	LangChain file tools, Pandas, SQLAlchemy	Read/write files, query databases
Communication	Gmail, Slack, Teams, SendGrid tools	Send emails, post messages, notify
APIs & Integrations	Composio (150+ apps), Zapier tools	GitHub, Linear, Jira, Salesforce, etc.
MCP Servers	Any MCP server (see Guide 2)	Standardized tool interface for any capability
Memory Tools	Mem0, Zep, LangChain memory	Remember user preferences and past interactions
Human-in-Loop	LangGraph interrupt, HITL patterns	Pause for human approval before acting

WHAT YOU CANNOT DO WITHOUT THE AGENTIC LAYER

Automate multi-step workflows autonomously. Interact with external systems dynamically. Handle tasks that require planning, branching, and adaptation at runtime. Without agents, every workflow must be hard-coded — no dynamic decision making.

09 Layer 9 — Memory Systems

LLMs have no persistent memory by default — each API call starts fresh. Memory systems give AI applications the ability to remember information across turns, sessions, and users.

Memory Type	Scope	Tool / Approach	Use Case
In-Context (Buffer)	Current session only	Append messages to prompt. ConversationBufferMemory.	Short conversations, simple chatbots
Window Buffer	Last N turns	Keep only the last K messages. ConversationBufferWindowMemory.	Ongoing chat where recent context matters
Summarization Memory	Long sessions	LLM summarizes conversation so far. ConversationSummaryMemory.	Long conversations that exceed context window
Episodic (Vector)	Cross-session	Embed and store past interactions. Retrieve by similarity.	Personalized assistants, long-running agents
Entity Memory	Cross-session	Extract and store named entities (people, places, preferences).	CRM-like agents that track entities
Knowledge Memory	Global	RAG over a knowledge base. Static or periodically updated.	Domain knowledge access for all users
Mem0 / Zep	Cross-session	Dedicated memory service — auto-extract and retrieve facts.	Production user memory for apps

WHAT YOU CANNOT DO WITHOUT MEMORY

Build personalized AI experiences. Maintain context across sessions. Let agents learn from past interactions. Create assistants that remember user preferences. Without memory, every conversation starts from zero — frustrating for users.

10 Layer 10 — Evaluation and Observability

Without evaluation, you are flying blind. Evaluation measures whether your AI system actually works. Observability lets you understand what is happening inside it.

Evaluation Frameworks

Tool	Focus	Key Metrics	Best For
RAGAS	RAG evaluation	Faithfulness, Answer Relevancy, Context Precision/Recall	RAG pipelines, LLM-as-judge, no manual labels
TruLens	LLM app evaluation	RAG triad + custom feedback functions	Full pipeline evaluation with dashboards
DeepEval	LLM testing	G-Eval, hallucination, bias, toxicity metrics	Unit testing LLM outputs in CI/CD
PromptFoo	Prompt testing	Test cases, regression, A/B prompt comparison	Prompt engineering, prompt CI/CD
Evals (OpenAI)	Model benchmarking	Custom eval harness for any task	Comparing models on your specific task
LangSmith Evals	Integrated evaluation	Human feedback, auto-eval, regression	If already using LangSmith for tracing

Observability and Tracing

Tool	What It Traces	Best For
LangSmith	Every LangChain call: tokens, latency, input/output, cost	LangChain/LangGraph apps — best tracing tool
Arize Phoenix	RAG traces, embedding drift, retrieval quality over time	Open-source tracing + evaluation dashboard
Langfuse	LLM traces, evals, prompt management, cost tracking	Multi-framework, open-source, self-hostable
OpenTelemetry + OTEL	Standard trace/span format for AI systems	Enterprise observability, custom dashboards
Weights & Biases (W&B;)	Training runs, fine-tuning experiments, model versions	ML training, experiment tracking
MLflow	Experiments, models, deployment tracking	MLOps, model registry, reproducibility

WHAT YOU CANNOT DO WITHOUT EVALUATION

Know if your system is improving or degrading. Debug why answers are wrong. Catch regressions when you change a prompt or model. Make data-driven decisions about chunking strategy, retrieval K, or embedding model. Flying blind in production.

11 Layer 11 — Deployment and Serving

AI applications need infrastructure to run in production — serving LLMs, managing API costs, scaling with load, and handling failures.

LLM Serving — Running Models

Tool	Type	Best For
Ollama	Local inference	Run open models locally. <code>ollama run llama3</code> . Zero cost.
vLLM	Self-hosted serving	High-throughput GPU serving. PagedAttention for efficiency. Production open-source.
LM Studio	Local GUI	No-code local LLM runner. Download and run models.
llama.cpp	CPU/GPU inference	Run quantized models on CPU or GPU. Very portable.
HuggingFace TGI	Self-hosted serving	Text Generation Inference. Deploy any HF model with optimizations.
Modal	Serverless GPU	Run GPU workloads serverless. Pay per second.
Replicate	Model hosting	Deploy and call open models via API. Managed.
Together AI	API	Open model inference API. Cheaper than OpenAI for OSS models.
Groq	LPU inference	Extremely fast LLM inference via custom LPU chips.
OpenRouter	API aggregator	One API for 100+ models from all providers.

Application Deployment

Tool	Type	Best For
FastAPI	API framework	Build REST APIs around your AI system. Standard for Python AI APIs.
Streamlit	UI framework	Rapid AI app UIs — chat, dashboards, demos. 10 lines to UI.
Gradio	UI framework	Model demos, Spaces hosting. Best for ML demos.
Docker	Containerization	Package AI apps with all dependencies. Reproducible deployment.
Kubernetes / EKS	Container orchestration	Scale AI services in production. Your team's current infra.
AWS Lambda / GCF	Serverless	Event-driven AI workflows. Pay per invocation.
LangServe	LangChain deployment	One-line deployment of LangChain chains as REST APIs.
Chainlit	Chat UI	Production-grade chat UI for AI assistants. LangChain/LlamaIndex integration.

WHAT YOU CANNOT DO WITHOUT DEPLOYMENT INFRASTRUCTURE

Serve your AI system to users. Scale beyond a single machine. Monitor production health. Manage costs at scale. Achieve reliable uptime. Without deployment, your AI lives only in notebooks.

12 Layer 12 — Workflow Automation and Integration

Workflow automation tools let you connect AI capabilities to existing business systems — triggering AI on events, routing outputs to downstream systems, and building AI-powered business processes without writing full applications.

Tool	Type	AI Support	Best For
n8n	Self-hosted automation	Native LangChain AI nodes, LLM, agents, RAG	AI workflows, self-hosted, developer-friendly
Zapier	Cloud automation	OpenAI actions, basic LLM calls	Non-technical users, quick integrations
Make (Integromat)	Cloud automation	HTTP → AI APIs, limited native support	Visual workflows, moderate complexity
Prefect	Data pipeline	Python-native, schedule and observe AI pipelines	MLOps, data ingestion pipelines, monitoring
Apache Airflow	Data pipeline	Python operators for any AI task	Enterprise data pipelines, complex DAGs
LangGraph + APIs	Code-based	Full LangChain/graph agent workflows	Complex conditional AI workflows in code
Temporal	Workflow engine	Durable execution for long-running AI agents	Long-running agents, retry logic, reliability

13 Full Stack — How Everything Connects

Here is how all 12 layers work together in a production RAG + Agent application:

Layer	Component	Failure Mode If Missing
12. Application	FastAPI + Streamlit + Docker	No way to serve users. System lives in a notebook forever.
11. Orchestration	LangChain / LlamaIndex / LangGraph	Manually wire every API call. No chains, no agents, no RAG.
10. Evaluation	RAGAS + LangSmith	Cannot measure quality. Regressions go undetected.
9. Agents / MCP	CrewAI / LangGraph + MCP Servers	LLM cannot act. Only single-turn Q&A; possible.
8. Memory	Mem0 / Redis / Vector memory	Every conversation starts fresh. No personalization.
7. Retrieval	Qdrant + HNSW + Re-ranker	LLM answers from memory only. Hallucinations on private data.
6. Embedding	sentence-transformers + BM25	Cannot convert documents to searchable vectors.

5. Chunking	RecursiveCharacterTextSplitter	Entire documents in context. Token budget exceeded, no precision.
4. Data	PostgreSQL + S3 + Vector DB	No persistent storage. State lost on restart.
3. Context Mgmt	tiktoken + PromptTemplate	Cannot structure prompts correctly. Token limits exceeded.
2. Foundation Model	GPT-4o / Claude / Llama	No reasoning engine. Nothing works at all.
1. Compute	GPU / Cloud API	Cannot run any model. The entire stack is inert.

KEY TAKEAWAY

The compute layer (GPU/API) is the foundation — without it nothing runs. The LLM is the brain — without it there is no reasoning. The vector DB + embedding layer is the memory — without it RAG is impossible. Orchestration is the glue — without it every component is an isolated island. Evaluation is the feedback loop — without it you cannot improve systematically.

14 Recommended Stacks by Use Case

Use Case	Recommended Stack
Learning / Prototyping	Ollama (local LLM) + Chroma + all-MiniLM + LangChain + Streamlit
Production RAG API	OpenAI / Claude API + Qdrant + hybrid BM25+dense + LangChain + FastAPI + LangSmith + RAGAS
Private / Air-gapped RAG	Llama 3 via vLLM + Qdrant + BGE-M3 + LlamaIndex + FastAPI (all on-premise)
Multi-Agent System	GPT-4o + LangGraph + Composio tools + Mem0 memory + LangSmith tracing
No-Code AI Automation	n8n + OpenAI nodes + Qdrant + Pinecone
KYC / Document AI (NBFC)	AWS Textract (OCR) + Claude Vision + Qdrant + LangChain + RAGAS + FastAPI + Redis cache
Fine-tuned Domain Model	Llama 3 / Mistral base + QLoRA via PEFT/TRL + HuggingFace Hub + vLLM serving
Hackathon (fast start)	OpenAI API + LangChain + Chroma (in-memory) + Streamlit + RAGAS